Week 5 SQLite Inspection Guide

Purpose

Use this guide to inspect the SQLite database created by Entity Framework Core during Week 5 Lab 1.

The guide helps you verify that:

- The migration created the intended schema.
- Primary keys, foreign keys, indexes, and nullability match the ERD.
- The seed data was inserted correctly.
- Data remains available after the application restarts.

This is an inspection activity. Continue to access application data through the controller, service, DAO, and `DbContext` rather than adding direct SQL queries to controllers or views.

Database Location

The Lab 1 connection string is:

```
{
  "ConnectionStrings": {
    "StudentManagementDatabase": "Data Source=student-management.db"
  }
}
```

A relative SQLite path is normally resolved from the working directory used to start the application.

If you cannot find the database, search the project for:

```
student-management.db
```

Do not assume that the database is stored beside `appsettings.json`. Its location depends on the working directory used by your IDE or `dotnet run` command.

Suitable Inspection Tools

You may use any SQLite-compatible tool approved by your tutor, including:

- DB Browser for SQLite
- A database tool included in Visual Studio or JetBrains Rider
- A SQLite extension for Visual Studio Code
- The `sqlite3` command-line program

The screenshots and menu names vary between tools. Focus on the schema and data rather than reproducing a particular interface.

Inspect the Initial Schema

Open `student-management.db` and confirm that it contains:

```
Student
Enrolment
__EFMigrationsHistory
```

`__EFMigrationsHistory` is maintained by Entity Framework Core. It records which migrations have been applied to the database.

Inspect Student

Confirm that `Student` contains columns equivalent to:

```
student_number
first_name
last_name
email_address
```

Check that:

- `student_number` is the primary key.
- The main text fields are required.
- The configured maximum lengths appear in the schema where supported.
- `email_address` has a unique index.

Inspect Enrolment

Confirm that `Enrolment` contains columns equivalent to:

```
enrolment_id
student_number
programme_code
start_date
expected_end_date
mode_of_study
enrolment_status
```

Check that:

- `enrolment_id` is the primary key.
- `student_number` is required.
- `expected_end_date` is optional.
- `enrolment_status` is stored as readable text.
- A foreign key relates `Enrolment.student_number` to `Student.student_number`.

Inspect with the SQLite Command Line

If `sqlite3` is installed, open the database from the directory containing it:

```
sqlite3 student-management.db
```

Display the available tables:

```
.tables
```

Display the `Student` schema:

```
.schema Student
```

Display the `Enrolment` schema:

```
.schema Enrolment
```

Display the migrations history:

```
SELECT *
FROM __EFMigrationsHistory;
```

Exit the command-line tool:

```
.quit
```

Inspect Keys and Indexes

Display the columns in `Student`:

```
PRAGMA table_info('Student');
```

Display the columns in `Enrolment`:

```
PRAGMA table_info('Enrolment');
```

Display the foreign keys defined on `Enrolment`:

```
PRAGMA foreign_key_list('Enrolment');
```

Display indexes on `Student`:

```
PRAGMA index_list('Student');
```

For each index returned, inspect its columns by replacing the placeholder with the actual index name:

```
PRAGMA index_info('<index_name>');
```

Confirm that the email-address index is unique.

Inspect the Seed Data

Display all students:

```
SELECT
    student_number,
    first_name,
    last_name,
    email_address
FROM Student
ORDER BY last_name, first_name;
```

Display all enrolments:

```
SELECT
    enrolment_id,
    student_number,
    programme_code,
    start_date,
    expected_end_date,
    mode_of_study,
    enrolment_status
FROM Enrolment
ORDER BY student_number, start_date;
```

Display the enrolments for `S100002`:

```
SELECT
    programme_code,
    start_date,
    expected_end_date,
    mode_of_study,
    enrolment_status
FROM Enrolment
WHERE student_number = 'S100002'
ORDER BY start_date;
```

The result should contain two enrolments ordered from earliest to latest.

Display students with no enrolments:

```
SELECT
    student.student_number,
    student.first_name,
    student.last_name
FROM Student AS student
LEFT JOIN Enrolment AS enrolment
    ON enrolment.student_number = student.student_number
WHERE enrolment.enrolment_id IS NULL;
```

This query is useful when testing the **No enrolments found** alternative flow through the web application.

Compare the Database with the ERD

Complete a short comparison using these questions:

1. Do the table and column names match the ERD?
2. Do the primary keys match?

3. Is the Enrolment-to-Student foreign key present?
4. Is the Student email-address index unique?
5. Is `expected_end_date` nullable?
6. Is `enrolment_status` stored as text?
7. Did Entity Framework Core add any technical objects not shown in the ERD?
8. Can every database difference be explained by the EF Core mapping or SQLite provider?

Do not change the migration merely because SQLite displays a type differently from the ERD. First determine whether the difference changes the intended behaviour or constraint.

Verify Through the Web Application

Database inspection confirms what was stored. The main use case must still be tested through the application.

Use the browser to verify that:

- Selecting **View enrolments** calls the controller action.
- The controller delegates to the service.
- The service delegates to `EnrolmentDao`.
- The DAO retrieves records through `StudentManagementDbContext`.
- The seeded enrolments appear in chronological order.
- A missing expected end date appears as **Not recorded**.
- A student with no enrolments produces **No enrolments found**.

Do not use the inspection tool as a replacement for testing the controller, service, DAO, and view.

Verify Persistence After Restart

1. Run the application and view the seeded enrolments.
2. Stop the application.
3. Start it again.
4. View the same enrolments.

5. Reopen the database if necessary.

Confirm that:

- Existing records remain available.
- The seeder does not create duplicate records.
- The migration history remains unchanged unless another migration was applied.
- The relationships remain valid.

Reset the Development Database

If the local database contains incorrect or partial test data, you may reset it during development:

```
dotnet ef database drop --force
```

Then recreate it:

```
dotnet ef database update
```

Start the application to run the seed process again.

Dropping the database deletes all local data. Use this command only with the Week 5 development database.

Inspection Checklist

- [ ] I located and opened `student-management.db`.
- [ ] I identified the three initial tables.
- [ ] I inspected both primary keys.
- [ ] I inspected the Enrolment foreign key.

- [ ] I confirmed that the Student email index is unique.
- [ ] I confirmed that `expected_end_date` is nullable.
- [ ] I confirmed that enrolment status is stored as text.
- [ ] I inspected the seeded Student and Enrolment rows.
- [ ] I confirmed that `S100002` has two chronologically ordered enrolments.
- [ ] I tested the enrolment page through the web application.
- [ ] I verified persistence after restarting the application.